TRINOVATECH
SMART TECHNOLOGY

École Polytechnique de Sousse
Génie Informatique
Spécialité Data Science & IA

RAPPORT DE STAGE D'ÉTÉ

ImmoInsight TN

Plateforme d'intelligence immobilière pour la Tunisie

Comment construire, à partir des seules annonces publiques, une lecture fiable et actualisée du marché immobilier tunisien, et estimer la valeur d'un bien avec une marge d'erreur mesurée honnêtement ?

RÉALISÉ PAR
Mohamed Aziz Karoui

ENTREPRISE D'ACCUEIL
STE TRINOVATECH

Année universitaire **2025 / 2026**

Résumé

Le marché immobilier tunisien est peu transparent : aucune base publique des prix de transaction n'existe, et les annonces en ligne sont dispersées, hétérogènes et souvent dupliquées. **ImmoInsight TN** reconstitue une lecture chiffrée de ce marché à partir des annonces publiques, pour la vente, la location à l'année et la location estivale. Un ordonnanceur collecte chaque jour les annonces de Tayara, Mubawab et Booking. Un pipeline ETL les nettoie, les déduplique et les géolocalise. La base compte 3 214 annonces exploitables, géolocalisées à 99,6 %.

Trois modèles de gradient boosting estiment le prix de vente, le loyer et le prix par nuit. Chacun n'est adopté que s'il bat une référence naïve : erreur médiane hors échantillon de 20,7 % en vente (référence 47,6 %), 22,0 % en location (28,8 %) et 29,4 % en estival (43,6 %), avec un intervalle à 80 % mesuré empiriquement. La plateforme s'utilise depuis une application Angular reliée à une API FastAPI : carte, statistiques, estimation, rentabilité, alertes et assistant conversationnel ancré dans les données. Elle est livrée en conteneurs Docker, couverte par 183 tests et validée par la Quality Gate de SonarQube.

Mots-clés : immobilier, Tunisie, web scraping, qualité des données, estimation automatique de valeur, gradient boosting, FastAPI, Angular, LLM, appel d'outils.

Abstract

ImmoInsight TN rebuilds a quantitative view of the Tunisian real-estate market from public listings (sales, long-term and holiday rentals). Listings are collected daily from Tayara, Mubawab and Booking, then cleaned, deduplicated and geocoded: 3 214 usable listings, 99.6 % geolocated. Gradient-boosting models estimate prices, rents and nightly rates. Each is adopted only if it beats a naive baseline, with median out-of-sample errors of 20.7 %, 22.0 % and 29.4 % and empirical 80 % intervals. A FastAPI backend and an Angular application provide maps, market statistics, valuation, yield simulation, alerts and an LLM assistant grounded in the data through tool calling. The platform ships as Docker containers, is covered by 183 tests, and passes the SonarQube Quality Gate.

Keywords: real estate, Tunisia, web scraping, data quality, automated valuation, gradient boosting, FastAPI, Angular, LLM tool calling.

Table des matières

Résumé et abstract i

1 Introduction 1
- 1.1 Contexte . . . 1
- 1.2 Problématique . . . 1
- 1.3 Objectifs . . . 1
- 1.4 Contributions . . . 2
- 1.5 Organisation du rapport . . . 2
- 1.6 Contexte du marché et état de l'art . . . 2

2 Architecture et choix techniques 4
- 2.1 Vue d'ensemble . . . 4
- 2.2 Choix technologiques . . . 4
- 2.3 Déploiement . . . 4
- 2.4 Modèle de données . . . 5

3 Collecte des données 7
- 3.1 Sources . . . 7
- 3.2 Techniques d'extraction . . . 7
- 3.3 Respect des sites et robustesse . . . 8
- 3.4 Ordonnancement . . . 8
- 3.5 Détection des annonces retirées . . . 8

4 Traitement et qualité des données 9
- 4.1 Le pipeline ETL . . . 9
- 4.2 Nettoyage et contrôles de qualité . . . 9
- 4.3 Déduplication en trois passes . . . 10
- 4.4 Géocodage . . . 10
- 4.5 Variables dérivées . . . 11

5 Modélisation 12
- 5.1 Protocole d'évaluation . . . 12
- 5.2 Encodage des localisations . . . 13
- 5.3 M1 — Prix de vente . . . 13
- 5.4 M2 — Loyer mensuel . . . 13
- 5.5 M3 — Prix par nuit en location estivale . . . 13
- 5.6 Synthèse des performances . . . 14
- 5.7 M4 — Score de fiabilité . . . 14
- 5.8 M5 — Analyse des descriptions . . . 15
- 5.9 Opportunités de marché . . . 15
- 5.10 Rentabilité locative . . . 15

6 API et application web : démonstration 16
- 6.1 L'API REST . . . 16
- 6.2 Conception de l'interface . . . 16
- 6.3 Carte des annonces . . . 17
- 6.4 Statistiques de marché . . . 18
- 6.5 Estimation et rentabilité . . . 19
- 6.6 Alertes de marché . . . 20
- 6.7 Usage sur téléphone . . . 21

7 Assistant conversationnel 22
- 7.1 Objectif et contraintes . . . 22
- 7.2 Architecture par appel d'outils . . . 22
- 7.3 Garde-fous . . . 23
- 7.4 Quotas et coûts . . . 23
- 7.5 Évaluation . . . 23

8 Qualité logicielle et déploiement 24
- 8.1 Tests automatisés . . . 24
- 8.2 Intégration continue . . . 24
- 8.3 Analyse statique avec SonarQube . . . 24
- 8.4 Conteneurisation . . . 25

9 Résultats, conclusion et perspectives 26
- 9.1 Bilan chiffré . . . 26
- 9.2 Principaux enseignements . . . 26
- 9.3 Limites . . . 26
- 9.4 Conclusion . . . 27
- 9.5 Perspectives . . . 27

Bibliographie 28

CHAPITRE 1

Introduction

1.1 Contexte

Acheter, louer ou investir dans un logement est l'une des décisions financières les plus lourdes d'un ménage. Dans les marchés où les prix de transaction sont publiés (registres notariaux, bases foncières ouvertes), les particuliers disposent d'outils d'estimation et d'indices de prix par quartier. Ce n'est pas le cas en Tunisie : l'information disponible au public se limite pour l'essentiel aux *annonces* publiées sur des sites généralistes ou spécialisés.

Ces annonces sont une source riche mais difficile à exploiter :

- elles sont **dispersées** entre plusieurs plateformes aux formats différents ;
- elles sont **hétérogènes et bruitées** : surfaces manquantes ou fantaisistes, prix saisis dans le mauvais champ, locations publiées en rubrique vente, localisation parfois réduite au gouvernorat ;
- elles sont **dupliquées** : un même bien est republié plusieurs fois, sur un ou plusieurs sites ;
- elles sont **éphémères** : un bien disparaît une fois vendu ou loué, et son prix peut évoluer entre-temps.

1.2 Problématique

Le projet répond à la question suivante :

> **Question directrice**
>
> Peut-on construire, à partir des seules annonces publiques, une plateforme qui donne une lecture *fiable, actualisée et compréhensible* du marché immobilier tunisien, et qui estime la valeur d'un bien avec une marge d'erreur mesurée honnêtement ?

Cette question se décompose en plusieurs sous-problèmes techniques : collecter durablement des données sur des sites qui n'ont pas été conçus pour cela ; transformer des annonces brutes en données cohérentes ; apprendre des modèles de prix sur un échantillon modeste, bruité et déséquilibré géographiquement ; présenter enfin les résultats à des utilisateurs non spécialistes sans masquer l'incertitude.

1.3 Objectifs

Les objectifs fixés au projet sont les suivants :

O1. **Collecter** automatiquement les annonces de vente, de location à l'année et de location estivale sur les principales sources, de manière régulière et respectueuse des sites.

O2. **Fiabiliser** les données : nettoyage, correction des incohérences, déduplication, géolocalisation et enrichissement par des points d'intérêt.

O3. **Estimer** le prix de vente, le loyer et le prix par nuit d'un bien, avec un intervalle de confiance et une erreur mesurée hors échantillon.

O4. **Détecter** les annonces suspectes et les opportunités (biens affichés nettement sous leur valeur estimée).

O5. **Restituer** l'information dans une application web : carte, statistiques, estimation, rentabilité, alertes et assistant conversationnel.

O6. **Industrialiser** la solution : tests automatisés, intégration continue, conteneurs Docker et analyse de qualité et de sécurité du code.

1.4 Contributions

Le travail réalisé aboutit à une plateforme complète et opérationnelle. Ses principaux apports sont :

- une **chaîne de collecte autonome** (neuf tâches planifiées, reprise après échec, pause automatique en cas de limitation de débit, détection des annonces retirées) ;
- un **pipeline de qualité des données** qui corrige plutôt qu'il ne supprime, trace chaque décision et publie un rapport par exécution ;
- des **modèles d'estimation évalués contre une référence naïve**, adoptés seulement s'ils la battent, avec un intervalle de prédiction empirique et un repli automatique sinon ;
- une **application web soignée et accessible**, construite sur une charte graphique dérivée d'un outil d'aide à la conception d'interfaces ;
- un **assistant conversationnel ancré dans les données**, qui interroge la plateforme par appel d'outils au lieu de produire des chiffres de mémoire ;
- une **industrialisation** complète : 183 tests, intégration continue, images Docker non-root, dépendances verrouillées par empreinte et Quality Gate SonarQube validée.

1.5 Organisation du rapport

Après l'état de l'art (section 1.6), le rapport présente l'architecture (chapitre 2), la collecte (chapitre 3), le traitement des données (chapitre 4) et la modélisation (chapitre 5), puis la plateforme (chapitre 6), l'assistant (chapitre 7), la qualité logicielle (chapitre 8) et enfin les résultats et perspectives (chapitre 9).

1.6 Contexte du marché et état de l'art

1.6.1 Le marché immobilier tunisien vu par ses annonces

Le marché tunisien présente trois segments aux logiques distinctes, que la plateforme traite séparément :

La vente. Les prix affichés reflètent la localisation (Grand Tunis, littoral du Cap Bon et du Sahel), la surface et le type de bien (appartement, villa, terrain). Les biens neufs vendus par des promoteurs côtoient les ventes entre particuliers.

La location à l'année. Les loyers mensuels sont plus homogènes que les prix de vente, mais dépendent fortement du quartier et de l'ameublement.

La location estivale. Concentrée sur le littoral (Hammamet, Sousse, Djerba, Mahdia…), elle se négocie à la nuit, avec une forte dispersion liée à la capacité, à la proximité de la plage et à la saison.

La nomenclature administrative ajoute une difficulté : le pays compte 24 gouvernorats découpés en délégations, mais les annonces mélangent gouvernorats, villes et quartiers (« Ennasr », « La Soukra », « Tunis »). La plateforme rattache donc chaque annonce à l'une de neuf *régions* homogènes (Grand Tunis, Cap Bon, Sahel, Sfax, Nord-Est, Nord-Ouest, Centre-Ouest, Sud-Est, Sud-Ouest) pour produire des statistiques comparables (§ 4.5).

Prix affichés, et non prix de transaction

Les annonces donnent un prix *demandé*, qui peut différer du prix finalement négocié. Toutes les statistiques et estimations du rapport portent sur des prix affichés. Elles décrivent l'offre visible en ligne, pas les transactions réalisées.

1.6.2 Estimation automatique de la valeur d'un bien

Prix hédoniques. L'approche classique de l'économie immobilière décompose le prix d'un bien en contributions de ses caractéristiques (surface, pièces, localisation, équipements) : c'est la méthode des prix hédoniques [1]. Une régression linéaire sur le logarithme du prix en est la forme la plus simple. Elle est interprétable, mais capture mal les interactions (l'effet d'une piscine dépend de la région) et les non-linéarités.

Modèles d'estimation automatique (AVM). Les *Automated Valuation Models* des grands portails immobiliers reposent aujourd'hui sur des ensembles d'arbres de décision entraînés sur des centaines de milliers de transactions. Les algorithmes de gradient boosting comme XGBoost [2] et LightGBM [3] dominent les données tabulaires : ils gèrent les valeurs manquantes, les interactions et les relations non linéaires, et peuvent être contraints à la monotonie (un bien plus grand ne doit pas valoir moins, toutes choses égales par ailleurs). La combinaison de plusieurs modèles par *stacking* [4] réduit encore la variance.

Encodage des localisations. Avec quelques centaines de quartiers, dont beaucoup ne comptent que quelques annonces, un encodage binaire (*one-hot*) est inutilisable. L'encodage par la cible lissé [5] remplace chaque zone par une moyenne pondérée entre la moyenne de la zone et la moyenne globale, d'autant plus proche de la moyenne globale que la zone est peu représentée (§ 5.2).

1.6.3 Qualité des données d'annonces

Déduplication. Repérer les annonces quasi identiques sans comparer toutes les paires est un problème classique de recherche de documents similaires. Le *MinHash* [6] estime la similarité de Jaccard entre ensembles de mots, et le hachage sensible à la localité (LSH) restreint les comparaisons aux paires candidates.

Détection d'anomalies. Sans étiquettes « annonce frauduleuse », la détection doit être non supervisée. L'*Isolation Forest* [7] isole les points atypiques par des partitions aléatoires : une annonce isolée en peu de coupes est suspecte. C'est la méthode retenue pour le score de fiabilité.

1.6.4 Grands modèles de langage et appel d'outils

Les grands modèles de langage (LLM) rédigent des réponses fluides mais peuvent *halluciner* des chiffres plausibles et faux, ce qui est rédhibitoire pour un service d'information sur les prix. Deux familles de solutions existent pour ancrer leurs réponses dans des données :

- la **génération augmentée par recherche** (RAG), qui insère dans le contexte des passages de documents retrouvés par similarité ; elle convient aux corpus textuels ;
- l'**appel d'outils** (*function calling*), où le modèle choisit une fonction et ses paramètres, le système l'exécute et renvoie un résultat exact que le modèle reformule [9, 10].

Les données de la plateforme étant structurées et chiffrées (prix, médianes, estimations), l'appel d'outils est l'approche adaptée : le modèle n'invente aucun nombre, il les lit dans les résultats d'outils (chapitre 7).

CHAPITRE 2

Architecture et choix techniques

2.1 Vue d'ensemble

La plateforme est organisée en couches indépendantes qui communiquent par la base de données (figure 2.1). Les collecteurs écrivent les annonces brutes ; le pipeline ETL les transforme en collections propres ; les modèles s'entraînent sur ces collections et y écrivent leurs scores ; l'API lit ces collections et les modèles pour servir l'application web et l'assistant. Grâce à ce découplage, un collecteur peut échouer sans affecter l'API, et l'ETL peut être rejoué à tout moment à partir des données brutes.

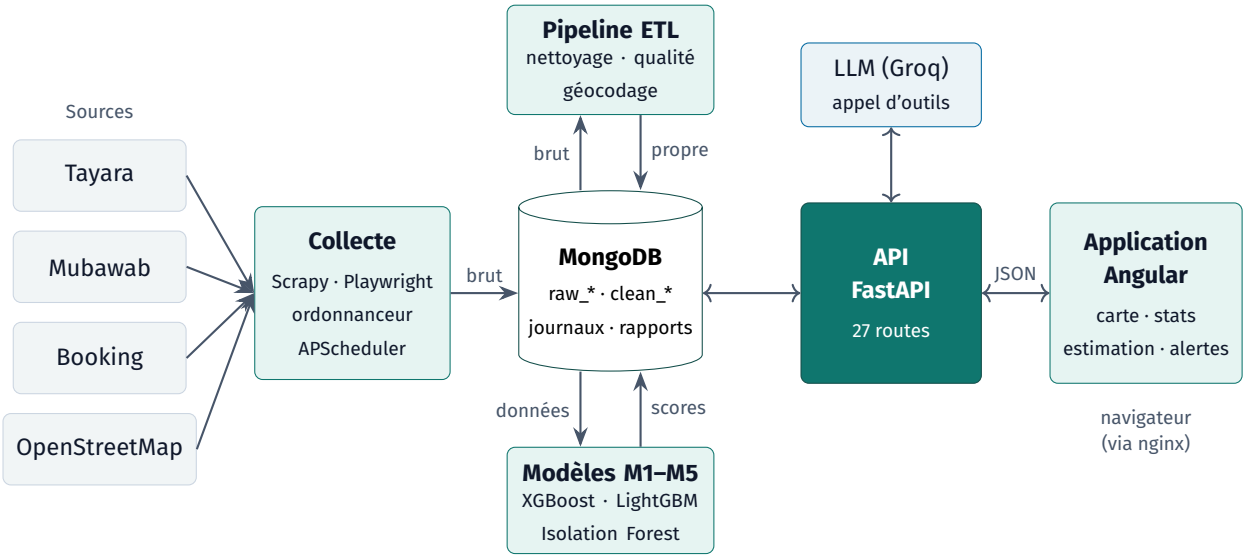


FIGURE 2.1. Architecture d'ensemble : les couches communiquent par la base de données.

2.2 Choix technologiques

Le tableau 2.1 résume la pile technique. Les choix ont été guidés par trois critères : la maturité de l'écosystème Python pour la donnée et l'apprentissage, la sobriété (aucun service payant indispensable, aucune GPU) et la facilité de déploiement [13].

2.3 Déploiement

La plateforme se lance par une seule commande, `docker compose up -d --build`, qui démarre quatre conteneurs (figure 2.2). Une même image backend sert l'API et l'ordonnanceur ; les modèles entraînés et le cache de géocodage sont montés en volumes partagés, ce qui permet à l'ordonnanceur de ré-entraîner les modèles chaque semaine sans reconstruire l'image. Les secrets (clé de l'API Groq) sont lus au démarrage depuis un fichier `.env`, jamais copiés dans l'image.

TABLE 2.1. Pile technique et justification des choix.

Couche	Technologie	Justification
Collecte	Scrapy 2.13, Playwright	Scrapy pour les sites rendus côté serveur (débit, reprise, middlewares) ; Playwright (Chromium sans interface) pour Booking, dont le contenu est produit en JavaScript.
Ordonnancement	APScheduler	Tâches planifiées en heure de Tunis, exécutées en sous-processus isolés avec délai maximal et journalisation.
Stockage	MongoDB 7	Documents de schéma variable selon la source ; agrégations pour les statistiques ; index géographiques.
Traitement	pandas, NumPy, datasketch	Nettoyage vectorisé ; MinHash/LSH pour les doublons textuels.
Géocodage	Photon, Nominatim	Services ouverts fondés sur OpenStreetMap, sans clé ; cache persistant.
Modèles	scikit-learn [8], XGBoost, LightGBM, spaCy	Gradient boosting sur données tabulaires, Isolation Forest, extraction de mots-clés en français.
API	FastAPI, Uvicorn	Validation des paramètres par typage, documentation OpenAPI générée, performances asynchrones.
Interface	Angular 17, Leaflet, Chart.js	Composants autonomes chargés à la demande ; cartographie avec regroupement de marqueurs et heatmap ; graphiques légers.
Assistant	Groq (gpt-oss-120b)	Inférence rapide, offre gratuite suffisante pour un prototype, appel d'outils natif ; modèle de secours en cas de quota.
Déploiement	Docker Compose, nginx	Quatre services (base, API, ordonnanceur, interface) ; nginx sert l'application et relaie l'API sur la même origine.
Qualité	pytest, GitHub Actions, SonarQube	183 tests, intégration continue, analyse statique de sécurité et de fiabilité.

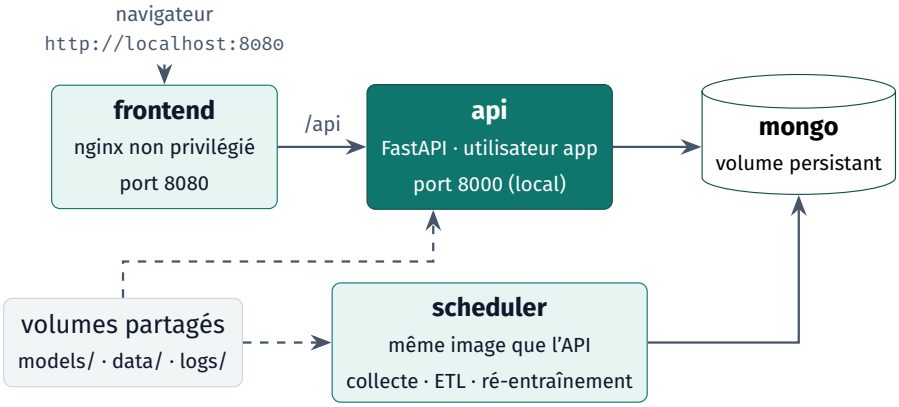


FIGURE 2.2. Déploiement avec Docker Compose : quatre services, deux images.

2.4 Modèle de données

MongoDB héberge une collection brute et une collection propre par segment, ainsi que des collections de service (tableau 2.2). Les collections brutes conservent tout ce qui a été collecté, y compris l'historique des prix observés ; les collections propres sont entièrement recalculées à chaque exécution de l'ETL, puis remplacées de façon atomique pour ne jamais exposer une collection vide ou partielle à l'API.

TABLE 2.2. Collections MongoDB et volumes en fin de stage (août 2026).

Collection	Documents	Contenu
`raw_vente` / `raw_location` / `raw_estival`	1 916 / 1 687 / 998	Annonces telles que collectées, historique des prix, statut (en ligne, retirée), date de dernière observation.
`clean_vente` / `clean_location` / `clean_estival`	1 141 / 1 275 / 798	Annonces nettoyées, dédupliquées, géolocalisées, enrichies (distances, région, score de fiabilité).
`geo_poi`	1 227	Points d'intérêt OpenStreetMap (plages, écoles, hôpitaux, transports…).
`estimations_oof`	2 356	Estimations hors échantillon des modèles, base des alertes d'opportunités.
`qualite_rapports`	—	Compteurs de chaque contrôle qualité, par exécution de l'ETL.
`scheduler_logs`	—	Début, fin, statut et sortie de chaque tâche planifiée.
`chat_logs`	—	Journal de l'assistant (question, réponse, outils appelés, modèle, durée) ; le visiteur n'y figure que par une empreinte de son adresse IP.

CHAPITRE 3

Collecte des données

3.1 Sources

Plusieurs sources ont été évaluées ; trois sont exploitées en production, complétées par OpenStreetMap pour les points d'intérêt (tableau 3.1). Les autres ont été abandonnées pour des raisons documentées : protection anti-robot persistante, domaine hors service ou données retirées par le fournisseur.

TABLE 3.1. Sources de données évaluées.

Source	Segments	Technique	Statut et remarques
Tayara	vente, location	Scrapy	**En production.** Données lues dans le JSON `__NEXT_DATA__` embarqué par le site (prix, surface, chambres, délégation) ; 47 % des annonces.
Mubawab	vente, location, estival	Scrapy	**En production.** Extraction par le balisage structuré JSON-LD `RealEstateListing`, puis par le bloc principal ; 29 %.
Booking	estival	Playwright	**En production.** Logements entiers uniquement (appartements, maisons de vacances, villas), 16 villes balnéaires ; 24 %.
OpenStreetMap	points d'intérêt	Nominatim	**En production.** [12] 1 227 points d'intérêt répartis en 8 catégories.
Afariat	vente, location	Playwright	Abandonné : protection Cloudflare (erreur 522) persistante.
Dar Tunisia	vente	—	Abandonné : domaine hors service.
Inside Airbnb	estival	téléchargement	Abandonné : la Tunisie a été retirée du jeu de données.

3.2 Techniques d'extraction

Privilégier les données structurées. Plutôt que d'extraire les valeurs du texte affiché, fragile et sujet aux confusions (un nombre de chambres pris pour un prix), les collecteurs lisent en priorité les données structurées que les sites embarquent pour leurs propres besoins : l'état initial de l'application Next.js chez Tayara, le balisage JSON-LD chez Mubawab. Le texte n'est utilisé qu'en dernier recours, avec des expressions régulières de complexité linéaire (§ 8.3).

Booking et le prix par nuit. Booking affiche le prix *total* d'un séjour (par exemple sept nuits). Une première version l'enregistrait comme prix par nuit, ce qui donnait une médiane absurde de 1 904 TND la nuit. Le collecteur lit désormais la durée du séjour sur la carte de résultat et calcule prix_nuit = prix_séjour/nb_nuits, tout en conservant les deux valeurs. Il extrait aussi la capacité (chambres, salles de bains, lits, surface), la note et le nombre d'avis, la distance au centre et à la plage. L'URL canonique (sans paramètres de dates) sert de clé pour éviter de créer un doublon à chaque passage.

Mise à jour incrémentale. Chaque annonce est identifiée par son URL. À chaque passage, la fonction d'enregistrement `upsert_annonce` :

- crée l'annonce si elle est nouvelle ;
- complète les champs manquants d'une annonce existante, sans écraser les valeurs déjà connues ;
- réécrit les champs volatils (note, nombre d'avis, prix du séjour) ;

- ajoute une entrée à l'historique des prix lorsque le prix a changé, ce qui alimente les alertes de variation.

3.3 Respect des sites et robustesse

La collecte doit durer sans dégrader les sites sources. Plusieurs mécanismes y contribuent :

- **débit limité** : une requête à la fois et 2,5 s d'intervalle pour Mubawab ; arrêt de la collecte incrémentale de Tayara après deux pages sans nouveauté ;
- **pause sur limitation** : un middleware Scrapy suspend le collecteur à la réception d'un code HTTP 429 ou 503, en respectant l'en-tête `Retry-After` s'il est présent, sinon avec une attente exponentielle ($30\,\text{s} \times 2^n$, plafonnée à 10 minutes, 5 tentatives) ;
- **isolation des tâches** : chaque tâche planifiée s'exécute dans un sous-processus avec un délai maximal. Un incident (un site qui ne répond plus, une page modifiée) n'interrompt donc pas l'ordonnanceur. Cela contourne aussi une contrainte de Scrapy, dont le moteur réseau ne peut être démarré qu'une fois par processus.

3.4 Ordonnancement

L'ordonnanceur exécute neuf tâches en heure de Tunis (tableau 3.2). Chaque exécution est journalisée (début, fin, statut, sortie) dans `scheduler_logs`, et une tâche manquée pendant que la machine était en veille est rattrapée dans un délai de six heures. L'ETL et le ré-entraînement attendent la fin des collectes en cours.

TABLE 3.2. Planning des tâches automatiques.

Tâche	Fréquence	Rôle
Tayara incrémental	tous les jours, 2 h	Nouvelles annonces, arrêt dès deux pages sans nouveauté (∼4 min).
Tayara complet	dimanche, 1 h	Parcours complet des catégories.
Mubawab	lundi et jeudi, 3 h	Trois pages par catégorie.
Mubawab — annonces sans prix	samedi, 3 h	Nouvelle visite des annonces dont le prix n'avait pas été lu.
Booking	mardi et vendredi, 4 h	16 villes balnéaires, logements entiers.
Vérification des retraits	tous les jours, 6 h	Nouvelle visite des annonces non revues depuis 14 jours.
Pipeline ETL	tous les jours, 6 h 45	Nettoyage, qualité, géocodage, fiabilité, opportunités.
Ré-entraînement M1–M3	dimanche, 9 h	Modèles réévalués et remplacés s'ils battent la référence.
Rapport d'alertes	tous les jours, 9 h 30	Synthèse des alertes du jour.

3.5 Détection des annonces retirées

Une annonce absente des dernières pages parcourues n'est pas forcément retirée : elle a pu simplement reculer dans la pagination. Déduire un retrait de cette seule absence produisait des faux positifs. La tâche de vérification visite donc à nouveau chaque annonce non revue depuis 14 jours : une réponse 404 ou 410, ou une redirection hors de l'annonce, confirme le **retrait** ; une réponse 200 la maintient **en ligne** ; une réponse 429 ou une erreur réseau laisse le statut **indéterminé** et met la source en pause. Le premier passage a confirmé 105 retraits et 45 annonces toujours en ligne ; les annonces retirées sont exclues des statistiques.

CHAPITRE 4

Traitement et qualité des données

4.1 Le pipeline ETL

Le pipeline transforme les collections brutes en collections propres en six étapes (figure 4.1). Il est exécuté chaque matin par l'ordonnanceur et peut être rejoué à tout moment : les collections brutes ne sont jamais modifiées par l'ETL, ce qui rend le traitement reproductible.

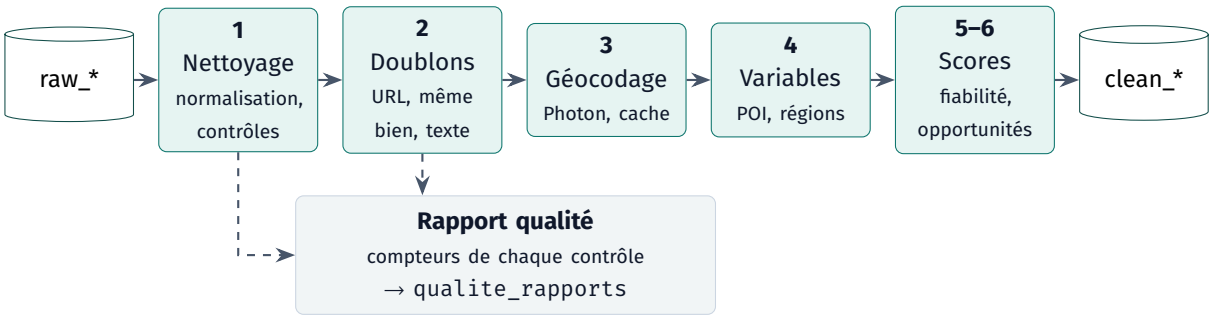


FIGURE 4.1. Étapes du pipeline ETL.

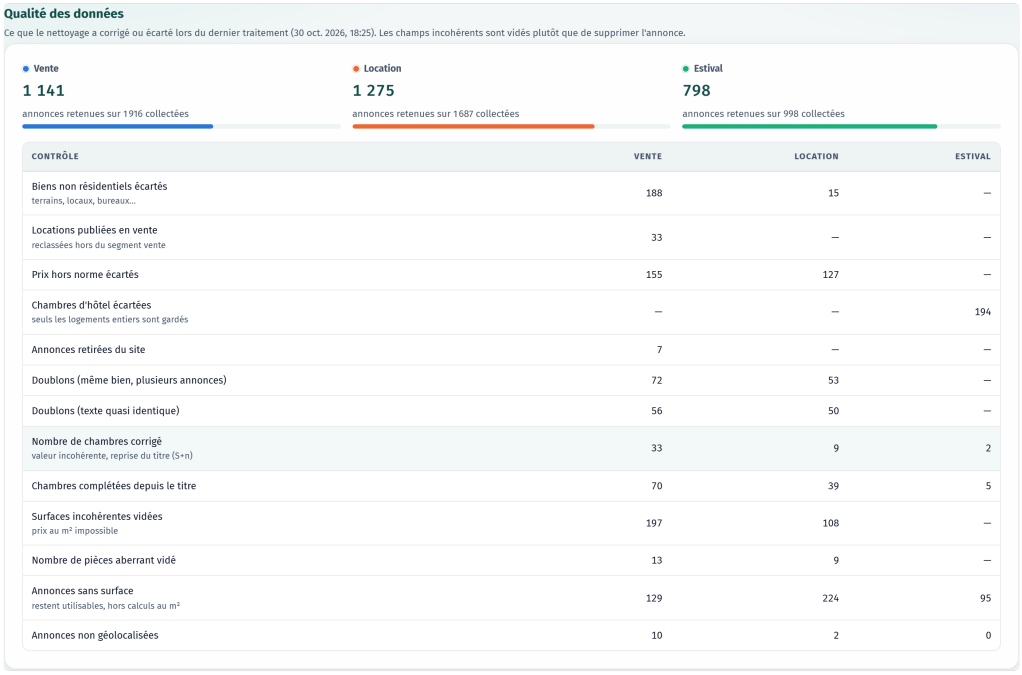
4.2 Nettoyage et contrôles de qualité

Normalisation. Les champs sont convertis dans des types et unités uniques : prix en dinars, surfaces en m², nombre de chambres selon la convention tunisienne « S+n », types de bien normalisés (appartement, villa/maison, terrain, maison de vacances), noms de villes sans accents ni variantes orthographiques. Les valeurs manquantes sont écrites `null`, jamais `NaN`. Un `NaN` transformé en texte avait en effet produit la requête de géocodage « nan, Tunisie », résolue en un bâtiment de Nabeul où s'empilaient 25 annonces de Bizerte, Sousse et Ariana.

Corriger plutôt que supprimer. Un audit des données a mis en évidence des erreurs récurrentes : des ventes à plus de dix chambres (le prix saisi dans le champ des chambres), des prix au m² impossibles, des locations publiées en rubrique vente, des chambres d'hôtel mêlées aux logements estivaux. La règle adoptée est de corriger la valeur fautive lorsque c'est possible, et de n'écarter l'annonce que si elle est inexploitable :

- un nombre de chambres aberrant est repris du titre (« S+3 ») ou vidé ; un nombre manquant est complété depuis le titre ;
- une surface est vidée (et non l'annonce supprimée) si le prix au m² qui en résulte sort des bornes plausibles : 200 à 15 000 TND/m² en vente, 2 à 80 TND/m²/mois en location ;
- un nombre de pièces supérieur à 15 est vidé ;
- les biens non résidentiels (locaux, bureaux, terrains en location) sont écartés, de même que les prix hors norme et, en estival, les chambres d'hôtel.

Chaque contrôle incrémente un compteur, et le rapport de chaque exécution est enregistré puis affiché dans l'application (figure 4.2). Au total, 60 % des annonces de vente collectées sont retenues (1 141 sur 1 916), 76 % en location (1 275 sur 1 687) et 80 % en estival (798 sur 998).



Qualité des données

Ce que le nettoyage a corrigé ou écarté lors du dernier traitement (30 oct. 2026, 18:25). Les champs incohérents sont vidés plutôt que de supprimer l'annonce.

Vente	Location	Estival
1 141	1 275	798
annonces retenues sur 1 916 collectées	annonces retenues sur 1 687 collectées	annonces retenues sur 998 collectées

CONTRÔLE	VENTE	LOCATION	ESTIVAL
Biens non résidentiels écartés terrains, locaux, bureaux…	188	15	—
Locations publiées en vente reclassées hors du segment vente	33	—	—
Prix hors norme écartés	155	127	—
Chambres d'hôtel écartées seuls les logements entiers sont gardés	—	—	194
Annonces retirées du site	7	—	—
Doublons (même bien, plusieurs annonces)	72	53	—
Doublons (texte quasi identique)	56	50	—
Nombre de chambres corrigé valeur incohérente, reprise du titre (S+n)	33	9	2
Chambres complétées depuis le titre	70	39	5
Surfaces incohérentes vidées prix au m² impossible	197	108	—
Nombre de pièces aberrant vidé	13	9	—
Annonces sans surface restent utilisables, hors calculs au m²	129	224	95
Annonces non géolocalisées	10	2	0

FIGURE 4.2. Rapport de qualité de la dernière exécution, tel qu'affiché dans l'onglet Statistiques.

4.3 Déduplication en trois passes

Un même bien est souvent republié, par le même annonceur ou par plusieurs agences. Les doublons faussent les médianes et surtout l'évaluation des modèles : un bien présent à la fois dans l'échantillon d'entraînement et dans l'échantillon de test rend l'erreur mesurée artificiellement faible. La déduplication procède en trois passes de plus en plus fines :

1. **même URL** : seule la version la plus récente est conservée ;
2. **même bien** : annonces de même prix, même surface, même ville et même nombre de chambres ; la fiche la plus complète est gardée, et les URL des doublons sont conservées dans un champ `doublons` ;
3. **texte quasi identique** : similarité de Jaccard estimée par MinHash et LSH [6], appliquée seulement aux descriptions d'au moins 80 caractères.

La troisième passe n'est pas appliquée à la location estivale : les descriptions de Booking sont courtes et standardisées, et deux logements distincts d'une même résidence y apparaissaient à tort comme doublons. Ce réglage a rétabli 125 logements estivaux injustement fusionnés. Au total, 128 doublons ont été retirés en vente et 103 en location.

4.4 Géocodage

La géolocalisation conditionne la carte, les distances aux points d'intérêt et les statistiques régionales. Les annonces n'ont pas de coordonnées : il faut les déduire d'une adresse souvent partielle. Le géocodeur procède ainsi :

1. construire la requête la plus précise possible à partir de la ville, de la délégation, de l'adresse affichée (Booking) ou de la localisation contenue dans l'URL Tayara (`/item/<catégorie>/<gouvernorat>/<délégation>/`) et dans le titre (« à La Manouba ») ;
2. interroger Photon (OpenStreetMap), limité à l'emprise de la Tunisie, avec Nominatim en secours ;
3. vérifier que le point trouvé est à moins de 40 km de la ville ou du gouvernorat annoncés, et rejeter sinon ;

4. ne jamais retomber sur une requête « Tunisie » seule, qui plaçait les annonces au centre géométrique du pays, en plein désert de Kébili.

Les résultats sont mis en cache de façon persistante et les appels limités à un par seconde, conformément aux conditions d'usage des services. Le type de résultat (adresse, rue, quartier, ville) est conservé : l'application signale une « position approximative » lorsque seul le centre d'une zone est connu. La couverture atteint 99,1 % en vente, 99,8 % en location et 100 % en estival.

4.5 Variables dérivées

Distances aux points d'intérêt. Pour chaque annonce géolocalisée, la distance au plus proche point d'intérêt de chacune des huit catégories (plages, écoles, universités, hôpitaux, commerces, pharmacies, restaurants, transports) est calculée par un arbre k-d sur coordonnées en radians, en distance du grand cercle. S'y ajoutent des indicateurs d'accessibilité (nombre de points d'intérêt dans un rayon de 5 km, distance moyenne) et un score d'équipements.

Régions. Le champ « gouvernorat » des annonces contient souvent une ville ou un quartier. Une table de correspondance ville/quartier → gouvernorat → région rattache chaque annonce à l'une des neuf régions ; 99 % des annonces sont classées (les autres n'ont aucune localisation).

Distribution des prix. La figure 4.3 montre la distribution du prix au m² obtenue après nettoyage. La médiane est de 2 533 TND/m² en vente et de 12,9 TND/m² par mois en location. Les deux distributions sont asymétriques à droite, ce qui justifie de modéliser le logarithme des prix et de résumer les statistiques par des médianes plutôt que des moyennes.

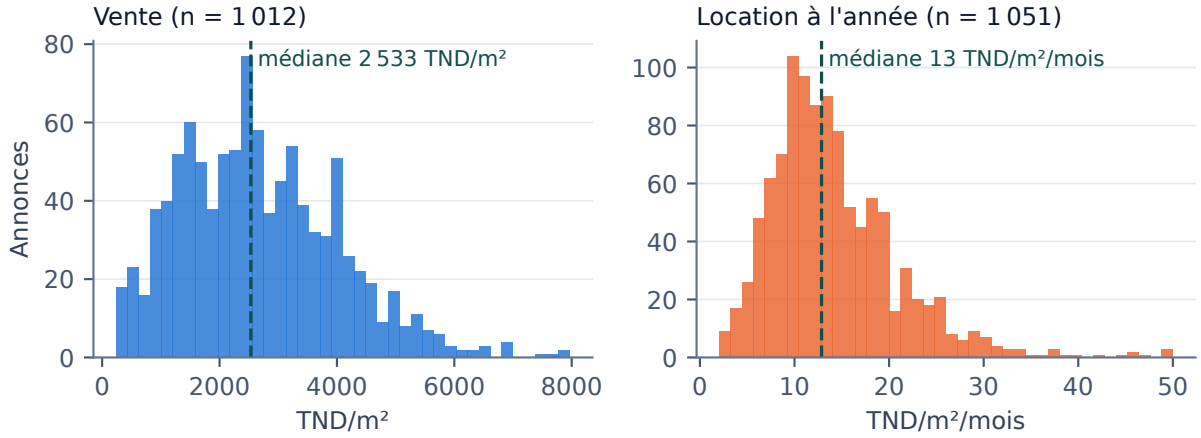


FIGURE 4.3. Distribution du prix au m² (vente) et du loyer au m² (location à l'année). Les queues au-delà de 8 000 et 50 TND/m² sont coupées pour la lisibilité ; les médianes portent sur toutes les annonces.

CHAPITRE 5

Modélisation

La plateforme comporte cinq modèles et un calculateur de rentabilité (tableau 5.1). Les trois modèles d'estimation (M1 à M3) partagent le même protocole d'évaluation, présenté en premier car il conditionne toutes les décisions de modélisation.

TABLE 5.1. Vue d'ensemble des modèles.

	Rôle	Méthode	Données
M1	Prix de vente	Stacking XGBoost + LightGBM, méta-modèle Ridge	1 051 ventes avec surface
M2	Loyer mensuel	LightGBM contraint	1 305 locations
M3	Prix par nuit (estival)	LightGBM contraint	671 logements entiers
M4	Fiabilité d'une annonce	Isolation Forest	1 141 ventes
M5	Mots-clés et standing	Règles et score pondéré	descriptions vente et estival
ROI	Rentabilité locative	Formules financières	saisies de l'utilisateur

5.1 Protocole d'évaluation

Métrique. Pour un utilisateur, une erreur de 50 000 TND n'a pas le même sens sur un studio que sur une villa. La métrique principale est donc l'**erreur relative médiane** hors échantillon :

$$\mathrm{EM} = \underset{i}{\text{médiane}} \left(\frac{|\hat{y}_i - y_i|}{y_i} \right) \times 100, \tag{5.1}$$

complétée par la part des estimations à moins de 20 % du prix réel et par l'erreur absolue moyenne. La médiane est préférée à la moyenne parce qu'elle n'est pas dominée par les quelques annonces atypiques qui subsistent après nettoyage.

Validation croisée répétée. Les erreurs sont mesurées par validation croisée à 5 plis (répétée 3 fois avec des partitions différentes pour M2 et M3) : chaque annonce est estimée par un modèle qui ne l'a jamais vue. Les variables qui dépendent de la cible (encodage des zones) sont recalculées *à l'intérieur* de chaque pli, faute de quoi l'évaluation serait optimiste.

Comparaison à une référence naïve. Une erreur de 20 % n'a de sens que comparée à ce qu'obtiendrait une méthode triviale. Chaque modèle est donc confronté à une **référence naïve** évaluée selon le même protocole : la médiane du prix au m² multipliée par la surface (M1), le barème de loyer au m² par zone (M2), la médiane du prix par nuit de la ville (M3).

> **Règle d'adoption**
>
> Un modèle n'est mis en service que s'il bat sa référence naïve en validation croisée. Sinon, la plateforme se replie automatiquement sur la référence. C'est arrivé pour M3 : avant que la collecte Booking ne soit enrichie (capacité, surface, distance à la plage), aucun modèle ne battait la médiane par ville, et c'est elle qui a été servie.

Intervalle de prédiction empirique. Une estimation ponctuelle sans marge est trompeuse. L'intervalle affiché est construit à partir des prédictions hors échantillon : on calcule le rapport $r_i = y_i / \hat{y}_i$ pour

chaque annonce, puis ses quantiles à 10 % et 90 %. L'intervalle à 80 % d'une nouvelle estimation $\hat{y}$ est

$$[\, q_{10}(r)\,\hat{y}\ ;\ q_{90}(r)\,\hat{y}\,]\,. \tag{5.2}$$

Pour M1, $q_{10} = 0{,}648$ et $q_{90} = 1{,}622$: un bien estimé à 300 000 TND a 80 % de chances d'être affiché entre 194 000 et 487 000 TND. Cette largeur, qui reflète honnêtement la dispersion des annonces, est affichée telle quelle à l'utilisateur. Une version antérieure affichait un intervalle fixe de ±10 %, sans fondement statistique ; il a été retiré.

5.2 Encodage des localisations

La localisation est le premier déterminant du prix, mais les zones sont nombreuses et souvent peu représentées. Chaque zone z (ville ou gouvernorat) est remplacée par une moyenne lissée du logarithme du prix au m² :

$$\mathrm{enc}(z) = \frac{n_z\,\bar{y}_z + k\,\bar{y}}{n_z + k}, \qquad k = 5, \tag{5.3}$$

où n_z est le nombre d'annonces de la zone, $\bar{y}_z$ leur moyenne et $\bar{y}$ la moyenne globale. Une zone à 2 annonces est ainsi rapprochée de la moyenne nationale, tandis qu'une zone à 80 annonces conserve son propre niveau. Une zone inconnue reçoit la moyenne globale, et l'API signale à l'utilisateur que la localisation n'a pas pu être prise en compte.

5.3 M1 — Prix de vente

Variables. Surface, nombre de pièces, de chambres et de salles de bains, étage, équipements (parking, ascenseur, piscine, gardien), type de bien, niveau de prix de la zone et distances aux plages, écoles, hôpitaux, transports et commerces. À la prédiction, si l'utilisateur ne fournit qu'une ville, les distances sont complétées par le *profil géographique* médian des annonces de cette zone.

Modèle. La cible est $\log(1 + \text{prix})$, ce qui stabilise la variance et interdit les prix négatifs qu'une première version produisait. Deux modèles de gradient boosting (XGBoost et LightGBM, 600 arbres, profondeur 5, taux d'apprentissage 0,04, régularisation L2) sont combinés par stacking : un méta-modèle Ridge est appris sur leurs prédictions hors échantillon [4]. Une **contrainte de monotonie** impose que le prix ne décroisse jamais avec la surface, toutes choses égales par ailleurs. Sans elle, le modèle pouvait estimer un appartement de 130 m² moins cher que le même bien de 120 m², ce qu'un utilisateur juge à juste titre absurde.

Résultats. Erreur médiane de **20,7 %** contre 47,6 % pour la référence, et $R^2 = 0{,}65$ sur le logarithme du prix. Le modèle divise donc par plus de deux l'erreur de la méthode naïve.

5.4 M2 — Loyer mensuel

Le loyer est modélisé par un LightGBM volontairement peu profond (arbres à 7 feuilles, profondeur 3, 400 itérations). Avec un millier d'annonces, des arbres profonds sur-apprennent. Les variables sont le logarithme de la surface, les chambres, les salles de bains, le type (villa), l'ameublement et le niveau de loyer de la ville et du gouvernorat. Le loyer est contraint à croître avec la surface et le nombre de chambres. Le modèle est comparé à deux références (tableau 5.2).

5.5 M3 — Prix par nuit en location estivale

Le prix par nuit est le segment le plus dispersé : pour une même ville, il varie avec la capacité, la proximité de la plage, le standing et la saison. Le modèle (LightGBM de même configuration que M2) utilise le niveau de prix de la ville, le type de logement, les chambres, la surface, le nombre

TABLE 5.2. M2 — comparaison au barème (validation croisée 5 plis × 3, 1 305 annonces).

Méthode	Erreur médiane	À moins de 20 %	Erreur abs. moyenne
LightGBM (retenu)	**22,0 %**	**45,8 %**	**866 TND**
Barème loyer au m² par zone	28,8 %	36,9 %	1 299 TND
Zone × surface	29,3 %	34,9 %	1 303 TND

de lits et de salles de bains, le front de mer et la distance à la plage ; le prix croît avec la capacité (chambres, surface, lits). Il obtient une erreur médiane de **29,4 %** contre 43,6 % pour la médiane par ville, et 36,5 % de ses estimations sont à moins de 20 % du prix affiché (22,5 % pour la référence).

5.6 Synthèse des performances

La figure 5.1 résume la comparaison des trois modèles à leur référence. Le gain est le plus net en vente, où la localisation et la surface expliquent une grande part du prix ; il est plus modeste en location, où le barème par zone est déjà une bonne approximation.

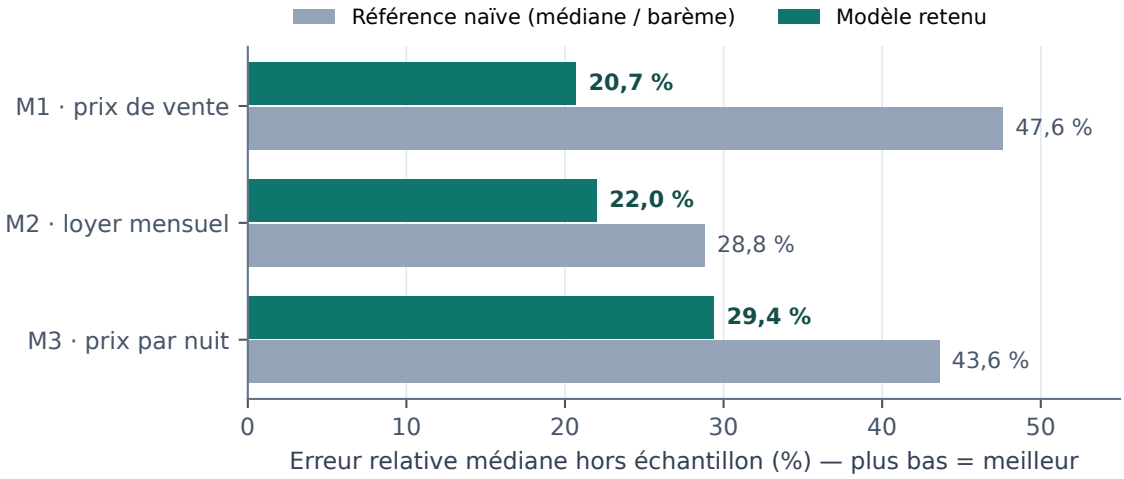


FIGURE 5.1. Erreur relative médiane hors échantillon des modèles retenus et de leur référence naïve.

5.7 M4 — Score de fiabilité

Faute d'annonces étiquetées comme frauduleuses, la détection est non supervisée. Une Isolation Forest [7] est entraînée sur trois variables standardisées : le prix, la surface et la distance aux transports. Les valeurs manquantes sont remplacées par les médianes d'entraînement, conservées pour la prédiction. La proportion d'anomalies attendue est fixée à 5 %. Le score d'anomalie s est converti en score de fiabilité sur 100 :

$$\text{fiabilité} = 100 \times \frac{s - s_{\min}}{s_{\max} - s_{\min}}, \tag{5.4}$$

avec les bornes $s_{\min}$ et $s_{\max}$ mesurées à l'entraînement. La fiabilité moyenne est de 81,7 ; les annonces de vente sous 30 sont signalées dans les alertes (28 au moment de la rédaction). Le calcul est intégré à l'ETL : chaque annonce reçoit son score, associé par identifiant et non par position. Une version antérieure associait les scores par position, si bien qu'une annonce pouvait recevoir le score d'une autre.

5.8 M5 — Analyse des descriptions

Le plan initial prévoyait d'affiner un modèle CamemBERT pour classer le standing. Sans données étiquetées ni GPU, une approche par règles a été retenue. Onze familles de mots-clés sont recherchées dans les descriptions par des listes d'expressions (vue mer, rénové, standing, proche plage, calme, lumineux, double vitrage, cuisine équipée, chauffage, sécurité, meublé). Un score pondéré en déduit une classe de standing :

$$S = 3\,\mathbf{1}_{\text{standing}} + 2\,\mathbf{1}_{\text{vue mer}} + \mathbf{1}_{\text{lumineux}} + \mathbf{1}_{\text{cuisine équipée}} + \mathbf{1}_{\text{double vitrage}} + \mathbf{1}_{\text{chauffage}}, \tag{5.5}$$

la classe valant *haut* si $S \geq 4$, *moyen* si $S \geq 2$ et *bas* sinon. Cette méthode est transparente et sans coût d'inférence, mais elle ne comprend ni la négation ni le contexte. Un classifieur appris est une perspective naturelle, dès qu'un échantillon étiqueté sera disponible.

5.9 Opportunités de marché

Une « opportunité » est une annonce affichée nettement sous l'estimation du modèle. Pour éviter qu'un modèle ne juge une annonce sur laquelle il a été entraîné, l'écart est calculé avec les **estimations hors échantillon** enregistrées pendant la validation croisée (`estimations_oof`). Le seuil n'est pas fixé arbitrairement : c'est le 10e percentile des écarts observés pour le modèle, soit −35 % en vente et −39 % en location. Une annonce n'est donc signalée que si son écart est plus défavorable que celui de 90 % des annonces. La plateforme recense 185 opportunités.

> **Une opportunité n'est pas une bonne affaire garantie**
> Un prix bas peut aussi trahir un défaut que les variables ne voient pas (travaux, titre foncier, erreur de saisie). L'alerte invite à examiner l'annonce, pas à acheter.

5.10 Rentabilité locative

Le calculateur de rentabilité combine location à l'année et location estivale :

$$\text{coût total} = \text{prix} \times (1 + 5\,\%) \quad \text{(frais d'acquisition)} \tag{5.6}$$

$$\text{revenu annuel} = 12 \times \text{loyer mensuel} + \text{nuits louées} \times \text{prix par nuit} \tag{5.7}$$

$$\text{rendement brut} = \text{revenu annuel} / \text{coût total} \tag{5.8}$$

$$\text{rendement net} = \text{rendement brut} - 0{,}5\,\% \quad \text{(taxe foncière)} \tag{5.9}$$

Le nombre de nuits louées vaut 90 par défaut (saison de juin à septembre) et se règle par un curseur. Un score sur 100 et une appréciation (de *déficitaire* à *excellent* au-delà de 8 % net) résument le résultat. Ce calcul est mis en perspective avec le rendement brut médian observé par ville, calculé uniquement sur les appartements et pour les villes comptant au moins 5 ventes et 5 locations ; il est de 5,1 % au niveau national.

CHAPITRE 6

API et application web : démonstration

Ce chapitre présente la plateforme du point de vue de l'utilisateur. Toutes les captures ont été réalisées sur l'application en fonctionnement, avec les données réelles collectées pendant le stage, par un navigateur piloté automatiquement (Playwright).

6.1 L'API REST

L'API FastAPI expose 27 routes sous `/api`, regroupées en huit domaines (tableau 6.1). Les paramètres sont validés par leur type et leurs bornes (`surface > 0`, `prix > 0`…). Une requête invalide reçoit une erreur 422 explicite, un modèle indisponible une erreur 503, au lieu d'une réponse 200 contenant un message d'erreur comme dans les premières versions. La documentation interactive OpenAPI est générée automatiquement (`/docs`).

TABLE 6.1. Domaines de l'API.

Préfixe	Routes	Rôle
`/api/annonces`	4	Liste paginée, annonces récentes, points de la carte, zones connues (autocomplétion).
`/api/{segment}/estimation`	3	Estimations M1, M2, M3 avec intervalle et erreur typique du modèle.
`/api/stats`	5	Indicateurs globaux, statistiques de marché (14 graphiques), rapport qualité.
`/api/alertes`	6	Nouvelles annonces, variations de prix, retraits, opportunités, résumé, planning.
`/api/fiabilite`	3	Scores M4 par annonce et alertes de fiabilité faible.
`/api/roi`	2	Calcul de rentabilité et classement des zones.
`/api/chat`	2	Assistant conversationnel et questions suggérées.
`/api/estival, /api/vente`	2	Villes balnéaires connues, prix moyen au m².

6.2 Conception de l'interface

L'interface est une application Angular 17 à composants autonomes. Chaque onglet est chargé à la demande (*lazy loading*), si bien que Leaflet et Chart.js ne pèsent pas sur le premier affichage : le paquet initial est passé de 698 à 232 ko. L'onglet courant est inscrit dans l'URL (`#stats`, `#estimation`…) : le rechargement, les favoris et le bouton Précédent conservent la section ouverte.

La charte graphique a été définie à l'aide de l'outil d'aide à la conception *UI/UX Pro Max*, qui recommande pour un produit immobilier une palette « confiance » (vert-bleu `#0F766E` et bleu `#0369A1`). Ses autres recommandations retenues sont les suivantes :

- une typographie de tableau de bord : Fira Sans pour le texte, Fira Code à chiffres de largeur fixe pour les montants, qui restent alignés dans les tableaux ;
- des icônes vectorielles cohérentes à la place des emojis ;
- une liste de contrôle d'accessibilité : contrastes d'au moins 4,5:1, contour de focus visible au clavier, lien d'évitement, zones cliquables de 44 px, respect de la préférence « réduire les animations » et information jamais portée par la seule couleur [11].

Les couleurs des segments (bleu pour la vente, orange pour la location, vert pour l'estival) et des types de bien sont identiques sur la carte et dans tous les graphiques. La palette des graphiques a

été choisie pour rester distinguable par les personnes daltoniennes.

6.3 Carte des annonces

La carte (figure 6.1) affiche les annonces géolocalisées de chaque segment. Les marqueurs sont regroupés en grappes numérotées qui s'ouvrent au zoom ; au zoom maximal, les annonces partageant la même position s'écartent en étoile (figure 6.2b). Des filtres par type de bien, nombre de chambres et budget s'appliquent instantanément, et la légende reprend les couleurs des types. La fiche d'une annonce donne le prix, la surface, le type, les chambres et le lieu, avec la mention « position approximative » si seul le centre de la zone est connu.

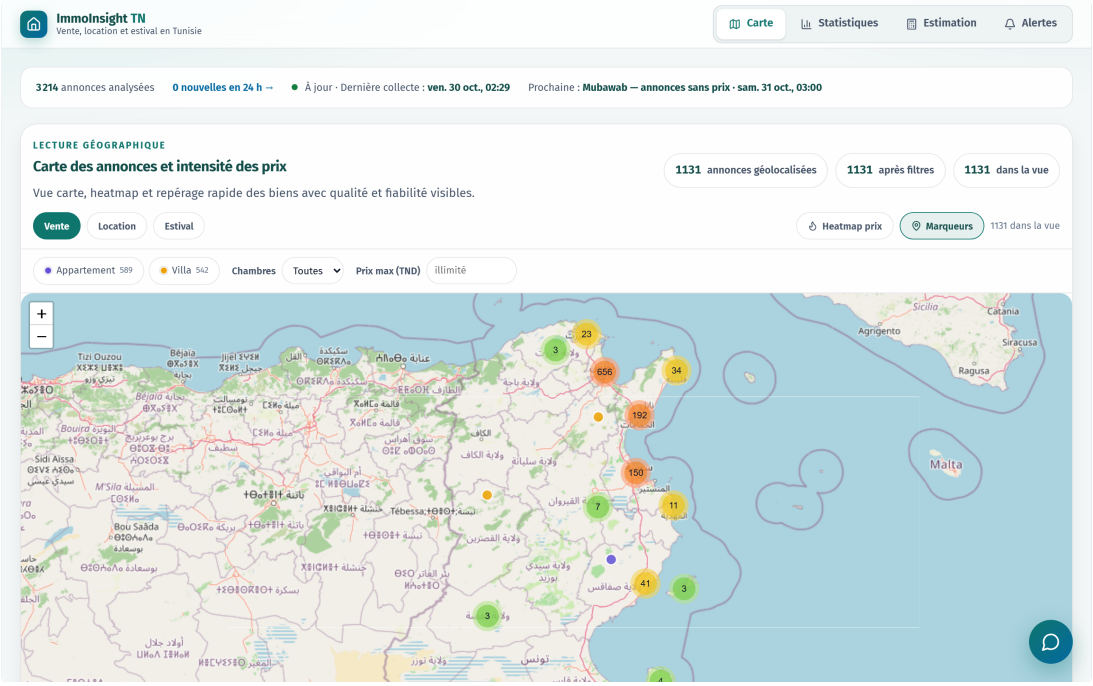
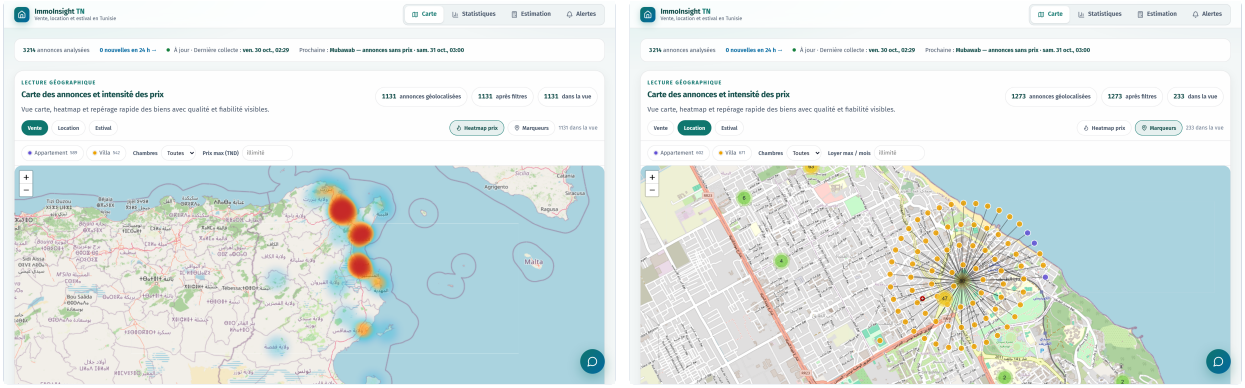


FIGURE 6.1. Carte des annonces de vente : grappes de marqueurs, filtres et barre d'état de la collecte.

Le mode *heatmap* (figure 6.2a) représente l'intensité des prix : prix au m² en vente, loyer en location, prix par nuit en estival. Les valeurs sont normalisées par percentiles, ce qui évite qu'une poignée de biens de luxe n'écrase toute l'échelle des couleurs. La concentration du marché sur le littoral est, de Bizerte à Sfax, immédiatement visible.



(a) Heatmap des prix au m² (vente). **(b)** Locations à La Marsa : grappe éclatée au zoom maximal.

FIGURE 6.2. Deux lectures de la carte.

6.4 Statistiques de marché

L'onglet Statistiques s'ouvre sur les indicateurs clés du marché (figure 6.3). Toutes les valeurs sont des médianes, peu sensibles aux biens d'exception. Une barre de navigation donne ensuite accès à huit sections thématiques et 14 graphiques. Chaque graphique dispose d'une vue « Données » qui affiche les chiffres sous forme de tableau, accessible aux lecteurs d'écran.

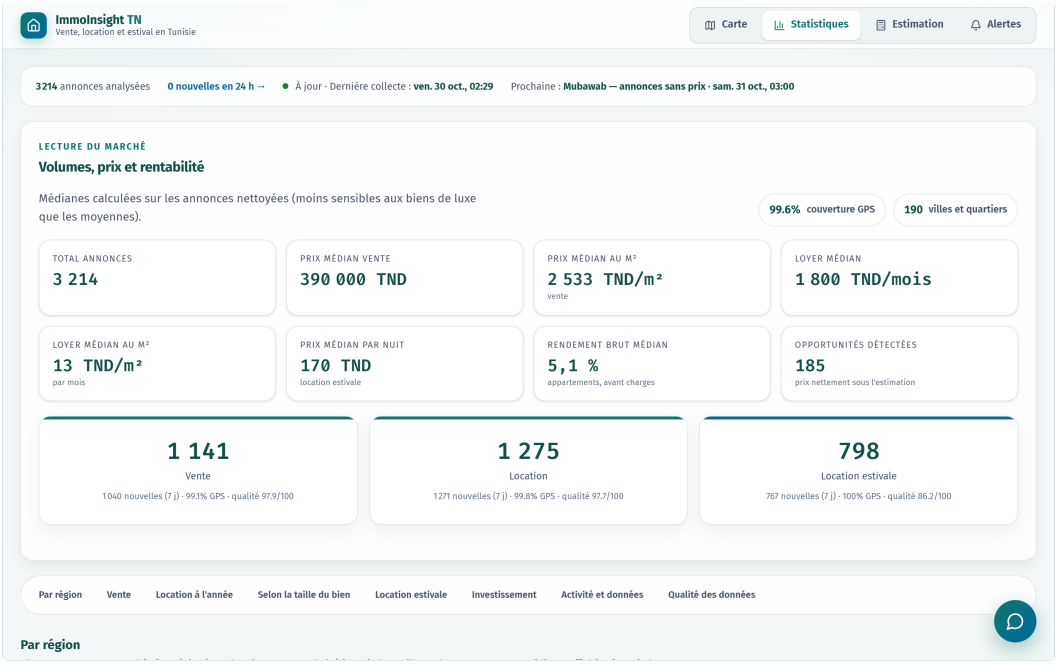


FIGURE 6.3. Indicateurs clés : volumes, prix et loyers médians, rendement brut et opportunités.

La section *Par région* (figure 6.4) compare les neuf régions. Le Grand Tunis concentre l'essentiel des annonces de location, le Cap Bon et le Sahel celles de l'estival. Le prix au m² le plus élevé se trouve dans le Sahel et le Grand Tunis, le loyer médian le plus élevé dans le Sud-Est. Les médianes ne sont affichées qu'à partir de 8 annonces, pour ne pas présenter comme représentatif un chiffre calculé sur trois biens.



FIGURE 6.4. Section « Par région » : volumes, prix au m², loyers, prix de vente et prix par nuit.

La section *Investissement* classe les villes par rendement locatif brut, calculé sur les seuls appartements à partir des médianes de prix et de loyer (tableau visible en bas de la figure 6.6). Ennasr, Aïn Zaghouan et Tunis offrent les rendements les plus élevés (5 à 6 %), les quartiers recherchés comme La Soukra ou Sahloul les plus faibles (autour de 4 %) : les prix d'achat y sont élevés par rapport aux loyers.

6.5 Estimation et rentabilité

L'onglet Estimation propose quatre modes : prix de vente, loyer mensuel, location estivale et rentabilité. Le formulaire et le résultat sont côte à côte (figure 6.5). Le champ de localisation propose les villes et quartiers effectivement présents dans les données. La validation se fait à la saisie, et les formulaires sont conservés pendant la session. Le résultat affiche :

- l'estimation et sa **fourchette probable à 80 %** (§ 5.1), représentée par une jauge ;
- l'**erreur typique du modèle**, mesurée hors échantillon (±21 % en vente) ;
- la **zone prise en compte**, ou un avertissement si elle est inconnue des données ;
- un bouton pour réutiliser le prix estimé dans le calcul de rentabilité.

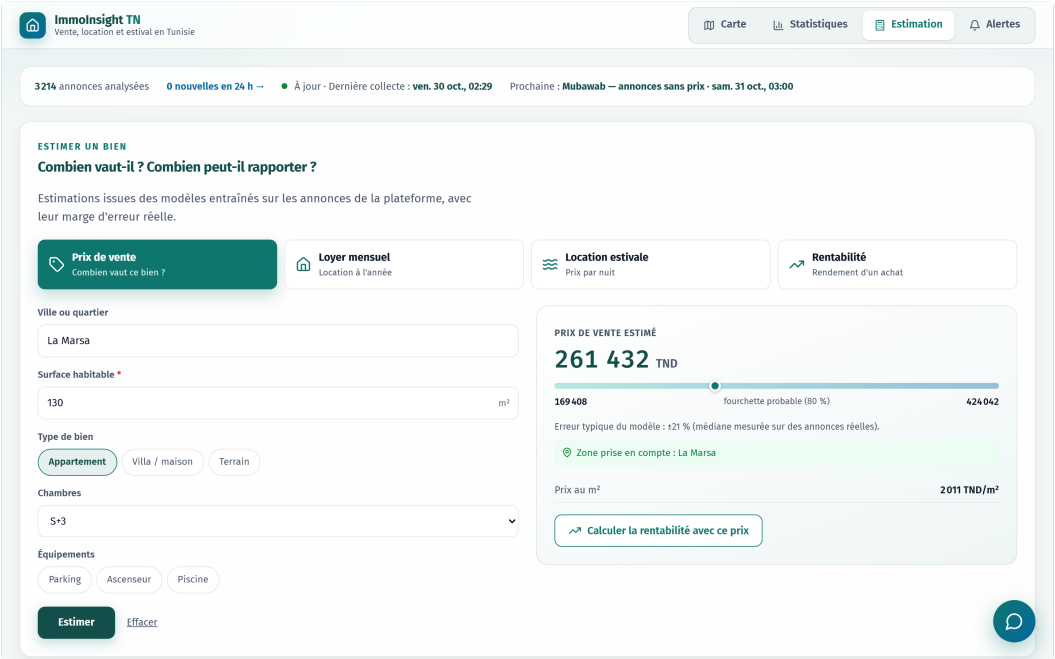


FIGURE 6.5. Estimation d'un appartement S+3 de 130 m² à La Marsa : 261 432 TND, fourchette à 80 % de 169 408 à 424 042 TND.

Le mode Rentabilité (figure 6.6) combine loyer à l'année et location estivale, avec un curseur pour le nombre de nuits louées. Le tableau des rendements médians par ville sert de point de comparaison : l'exemple simulé (11,1 % net) dépasse nettement le rendement médian observé (5,1 % brut), parce qu'il cumule les deux modes de location.

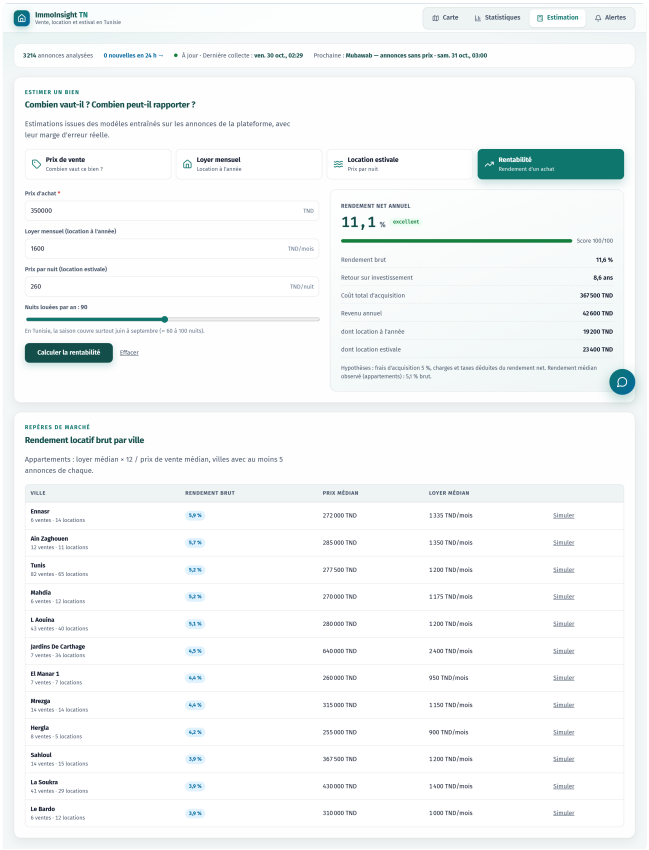


FIGURE 6.6. Simulation de rentabilité d'un achat à 350 000 TND, loué 1 600 TND par mois et 260 TND la nuit pendant 90 nuits.

6.6 Alertes de marché

L'onglet Alertes (figure 6.7) synthétise l'activité du marché sur une période (24 h, 7 ou 30 jours) et un segment. Cinq indicateurs cliquables donnent accès aux nouvelles annonces, aux variations de prix, aux opportunités, aux annonces retirées et aux annonces de fiabilité faible. La recherche et le tri s'exécutent côté serveur ; pendant leur mise à jour, le tableau reste affiché en estompé, sans clignotement ni perte de focus.

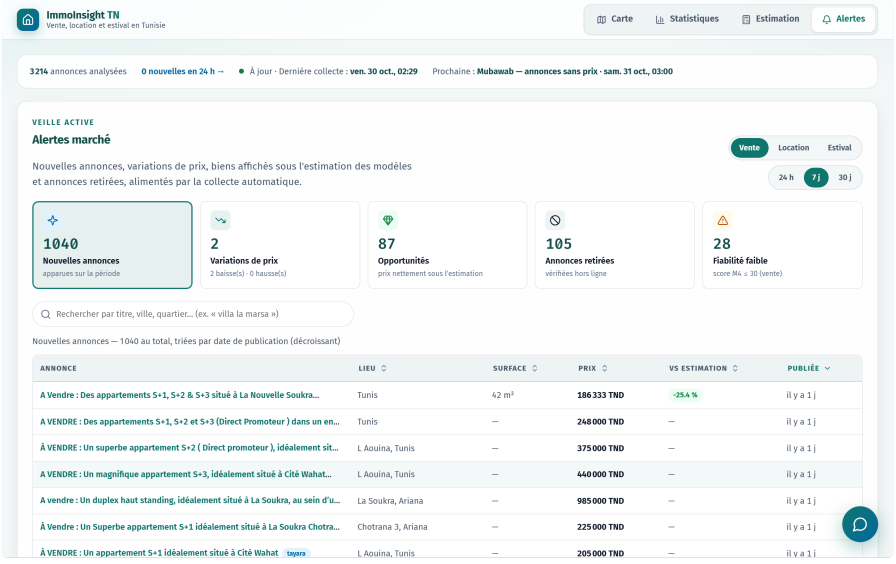
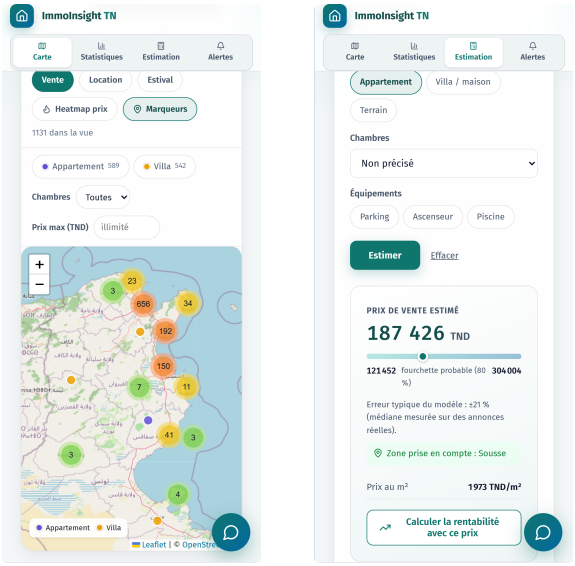


FIGURE 6.7. Alertes de la semaine en vente : 1 040 nouvelles annonces, 87 opportunités, 105 retraits confirmés.

La vue Opportunités compare chaque prix affiché à l'estimation hors échantillon du modèle. Les écarts les plus marqués (au-delà de −60 %) concernent souvent des annonces atypiques (maisons anciennes, biens à rénover, prix incomplet) : la vérification humaine reste indispensable, comme le rappelle le chapitre 5.

6.7 Usage sur téléphone

L'interface s'adapte aux petits écrans (figure 6.8). Les onglets passent sous le titre avec une icône au-dessus de leur libellé, les formulaires occupent toute la largeur, le résultat se place sous le formulaire, et aucune page ne défile horizontalement.



(a) Carte. **(b)** Estimation à Sousse.

FIGURE 6.8. Affichage sur téléphone (390 px de large).

CHAPITRE 7

Assistant conversationnel

7.1 Objectif et contraintes

L'assistant permet à tout visiteur d'interroger la plateforme en langage naturel, principalement en français : « Quel est le loyer médian d'un S+2 à La Marsa ? », « Combien vaut un appartement de 120 m² à Ennasr ? », « Quelles opportunités à la vente cette semaine ? ». Trois exigences ont guidé sa conception :

1. **aucun chiffre inventé** : chaque prix, médiane ou estimation doit provenir des données ou des modèles de la plateforme ;
2. **transparence** : la réponse cite ses critères, le nombre d'annonces utilisées, la date de mise à jour et, pour une estimation, sa fourchette ;
3. **sobriété** : l'offre gratuite du fournisseur doit suffire à un usage de démonstration.

7.2 Architecture par appel d'outils

Les données étant structurées, l'assistant ne recourt pas à la recherche documentaire (RAG). Il utilise l'**appel d'outils** : le modèle de langage choisit un outil et en remplit les paramètres, le serveur exécute la requête en lecture seule et renvoie un résultat exact et compact, puis le modèle rédige la réponse (figure 7.1). Le modèle n'écrit jamais de requête de base de données et ne voit que les résultats des outils.

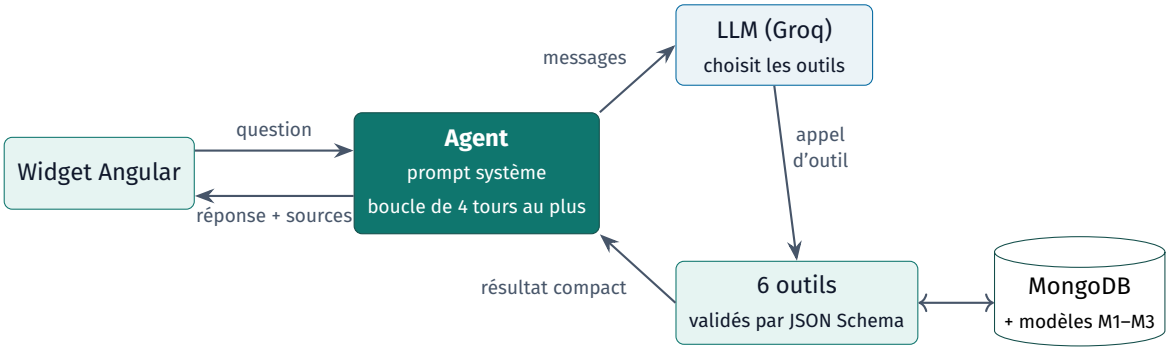


FIGURE 7.1. Boucle de l'assistant : le LLM choisit les outils, le serveur les exécute.

Six outils sont exposés au modèle (tableau 7.1). Leurs arguments sont décrits et validés par un schéma JSON strict (types, bornes, valeurs énumérées, aucune propriété supplémentaire) ; en cas d'erreur, le message est renvoyé au modèle pour qu'il corrige son appel.

TABLE 7.1. Outils disponibles pour l'assistant.

Outil	Rôle
`rechercher_annonces`	Annonces filtrées (segment, ville, type, chambres, budget), avec liens sources.
`statistiques_marche`	Médianes et volumes par ville, type ou nombre de chambres.
`estimer_prix`	Estimation M1, M2 ou M3 avec fourchette et erreur typique.
`alertes_marche`	Nouvelles annonces, variations de prix, opportunités, retraits.
`etat_plateforme`	Volumes, date de la dernière collecte et de la prochaine.
`calculer_rentabilite`	Rendement brut et net, retour sur investissement.

7.3 Garde-fous

- **Ancrage** : le prompt système impose de ne répondre qu'à partir des résultats d'outils, et de répondre « je n'ai pas cette information » sinon. Les questions hors sujet sont déclinées.
- **Données non fiables** : les textes d'annonces sont tronqués, débarrassés des caractères de contrôle et présentés au modèle comme des données à ne pas interpréter comme des instructions, ce qui le protège de l'injection de consignes.
- **Prompt système protégé** : le client ne peut envoyer que des messages `user` et `assistant` (2 000 caractères au plus, 10 derniers messages conservés).
- **Repli sur les zones** : si le modèle attribue une ville au mauvais gouvernorat (par exemple Ennasr « à Tunis »), l'outil relâche le filtre au lieu de répondre qu'il n'y a aucune annonce.
- **Rendu sûr** : la mise en forme Markdown du widget est appliquée sur un texte préalablement échappé, ce qui exclut toute injection HTML.

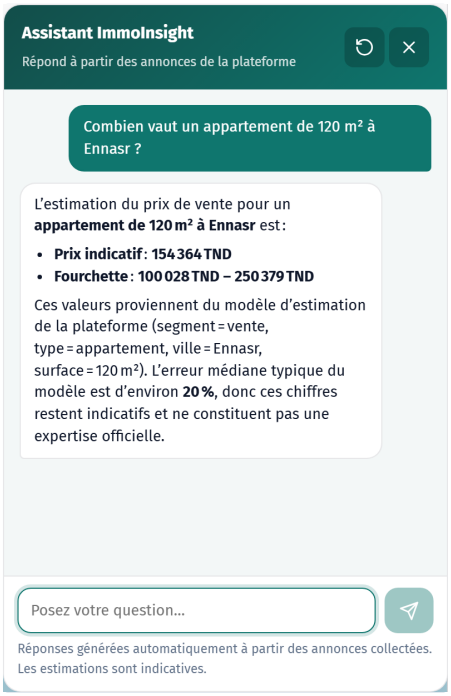


FIGURE 7.2. Réponse de l'assistant à une demande d'estimation.

7.4 Quotas et coûts

L'offre gratuite de Groq limite chaque modèle à environ 8 000 jetons par minute et 1 000 requêtes par jour. Plusieurs mécanismes permettent de rester dans ces limites :

- basculement immédiat sur un second modèle en cas de quota atteint ou d'appel d'outil mal formé ;
- cache de 15 minutes pour les questions sans historique, notamment les suggestions ;
- limite de 20 questions par tranche de 10 minutes et par visiteur.

Une ouverture au grand public nécessiterait l'offre payante.

7.5 Évaluation

Un jeu de 14 questions types couvre tous les outils, les questions hors sujet et les cas limites (ville inconnue, zone mal rattachée). Pour chaque question, le script d'évaluation vérifie l'outil appelé et compare les chiffres de la réponse aux valeurs calculées directement par les outils. Il contrôle aussi qu'un refus ne contient aucun prix (tableau 7.2).

TABLE 7.2. Évaluation de l'assistant sur 14 questions types.

Modèle	Réussite	Jetons / question	Durée / question
openai/gpt-oss-120b (principal)	**14 / 14**	~ 3 300	~ 1,6 s
qwen (secours de quota)	7 / 14	—	—

Les échecs du modèle de secours proviennent de quotas atteints et d'appels d'outils mal formés ; il n'est donc utilisé qu'en dernier recours. La figure 7.2 montre une réponse type, avec fourchette, critères et erreur typique.

CHAPITRE 8

Qualité logicielle et déploiement

8.1 Tests automatisés

La suite compte **183 tests** pytest, organisés par domaine (tableau 8.1). Les tests unitaires portent sur des fonctions pures (extraction d'une carte Booking, normalisation, règles de qualité, calcul de rentabilité). Les tests d'intégration interrogent l'API et les outils de l'assistant sur la base réelle ; ils sont ignorés automatiquement lorsqu'aucune base MongoDB n'est joignable. Les appels au modèle de langage sont simulés, ce qui rend les tests de l'assistant déterministes et gratuits.

TABLE 8.1. Organisation des tests.

Fichier	Couverture
`test_scrapers.py`	Extraction Tayara, Mubawab et Booking sur des pages de référence, URL canoniques, prix par nuit.
`test_etl.py`	Nettoyage, filtre des locations publiées en vente, géocodage (contrôle de distance), données de la carte.
`test_qualite.py`	Corrections de chambres et de surfaces, bornes de prix au m², déduplication, middleware de limitation.
`test_regions.py`	Rattachement ville → gouvernorat → région.
`test_api.py`	Routes, validation des paramètres, codes d'erreur 422/503, estimations, statistiques.
`test_alertes.py`	Nouvelles annonces, variations, retraits, opportunités, recherche et tri.
`test_chatbot.py`	Outils sur la base réelle, schémas JSON, boucle de l'agent et endpoint avec LLM simulé.

8.2 Intégration continue

À chaque envoi sur le dépôt, GitHub Actions exécute trois tâches :

1. **backend** : installation des dépendances verrouillées, vérification que l'API démarre sans base, sans modèles et sans clé, puis exécution des tests (113 réussis et 70 ignorés sans base, en une quinzaine de secondes) ;
2. **frontend** : build de production de l'application Angular ;
3. **docker** : construction des deux images, une fois les deux premières tâches réussies.

Un nouvel envoi sur la même branche annule l'exécution précédente.

8.3 Analyse statique avec SonarQube

Une analyse SonarQube a signalé 114 points, dont des notes D en sécurité et en fiabilité. Ils ont été traités par ordre de gravité, jusqu'à franchir la *Quality Gate*, qui exige une note A en sécurité sur le nouveau code. Les corrections principales sont les suivantes.

Sécurité.

- **Désactivation de la protection XSS (bloquant)** : les icônes SVG étaient injectées en contournant la protection d'Angular. Elles sont désormais écrites dans le gabarit du composant, et la protection reste active partout.
- **Dépendances non verrouillées** : un fichier `requirements.lock` fixe la version exacte et l'empreinte

SHA-256 des 120 paquets. Ils sont installés avec `--require-hashes` et `--only-binary :all:`, ce qui empêche l'installation d'un paquet altéré ou l'exécution d'un script d'installation.

- **Scripts npm** : `npm ci --ignore-scripts` et `npm run build` à la place de `npx`, qui pouvait télécharger et exécuter un paquet à la volée.
- **Images exécutées en root** : l'API tourne sous un utilisateur dédié `app`, et l'interface est servie par la variante non privilégiée de nginx.
- **Copie récursive dans l'image** : seuls les dossiers de code exécuté sont copiés, ce qui exclut tout fichier sensible du contexte de construction.

Fiabilité.

- **Expressions régulières à coût super-linéaire** : une recherche de la forme `(\d+)\s*m²` réessaie chaque position d'une longue suite de chiffres, ce qui donne un coût quadratique. Elles sont remplacées par une fonction qui parcourt chaque nombre une seule fois puis teste l'unité à la position suivante. Le temps est linéaire et les résultats sont identiques sur les cas réels.
- Tris de dates par `localeCompare`, regex de fuseau horaire parenthésées, détection des valeurs non numériques par `math.isnan` au lieu de `v == v`.

Les points restants relèvent de la maintenabilité : fonctions jugées trop complexes et assertions de tests composées. Ils sont listés comme pistes d'amélioration.

8.4 Conteneurisation

Image backend. Basée sur Python 3.13, elle contient le code, les dépendances verrouillées et le navigateur Chromium utilisé par Playwright, installé dans un dossier lisible par l'utilisateur non privilégié. Un contrôle de santé interroge `/health` toutes les 30 secondes.

Image frontend. Construite en deux étapes : l'application Angular est compilée dans une image Node, puis seuls les fichiers statiques sont copiés dans l'image nginx. nginx relaie `/api` vers le conteneur de l'API. L'application et l'API partagent ainsi la même origine, ce qui supprime toute configuration CORS. La limite de questions de l'assistant reste appliquée par visiteur, grâce à l'adresse IP réelle transmise par le relais.

Vérifications. La pile complète a été démarrée et la base restaurée par `mongodump`/`mongorestore` (11 419 documents). L'interface et l'API fonctionnent via nginx. Dans les images non privilégiées, l'API écrit dans les volumes de données et de journaux et lance bien Chromium.

CHAPITRE 9

Résultats, conclusion et perspectives

9.1 Bilan chiffré

Le tableau 9.1 rassemble les principaux résultats, et les met en regard de l'état du projet en début de stage (juillet 2026), avant la refonte de la collecte et des modèles.

TABLE 9.1. Évolution des principaux indicateurs.

Indicateur	Juillet 2026	Août 2026
Annonces de vente exploitables	60	1 141
Annonces de location exploitables	4	1 275
Logements estivaux exploitables	186	**798**
Couverture de géolocalisation	—	**99,6 %**
Erreur médiane M1 (vente)	$R^2 \approx 0{,}09$ (77 biens)	**20,7 %**
Erreur médiane M2 (loyer)	barème fixe	**22,0 %**
Erreur médiane M3 (estival)	non comparée	**29,4 %**
Collecte	manuelle	**9 tâches planifiées**
Routes de l'API	21	**27**
Tests automatisés	43	**183**
Déploiement	local	**Docker, CI, Quality Gate**

9.2 Principaux enseignements

La qualité des données domine la qualité des modèles. Les gains les plus importants ne sont pas venus des algorithmes mais des données. Le prix de séjour Booking lu comme un prix par nuit, des prix et surfaces multipliés par dix, des coordonnées empilées à Nabeul, des doublons gonflant artificiellement les performances : chacun de ces défauts rendait toute modélisation vaine. La traçabilité systématique (compteurs de qualité, rapports par exécution) a permis de les repérer.

Évaluer contre une référence change les décisions. Comparer chaque modèle à une méthode triviale a évité de mettre en service des modèles plus complexes mais pas meilleurs. Ce fut le cas de M3 tant que les caractéristiques des logements n'étaient pas collectées. Le chiffre affiché à l'utilisateur (erreur typique, fourchette à 80 %) découle directement de cette évaluation.

Rendre l'incertitude visible. Une fourchette de 169 000 à 424 000 TND peut sembler large. Elle reflète pourtant la réalité : des biens de même surface dans le même quartier sont affichés à des prix très différents. Afficher une précision illusoire aurait été plus séduisant, mais trompeur.

9.3 Limites

- **Prix affichés.** Les données portent sur des prix demandés, pas sur des prix de transaction. L'écart, inconnu, varie probablement selon le segment et la conjoncture.
- **Couverture inégale.** Le Grand Tunis, le Cap Bon et le Sahel concentrent les annonces ; le Centre-Ouest et le Sud-Ouest en comptent trop peu pour produire des médianes fiables. Les estimations y reposent largement sur la moyenne nationale.
- **Historique court.** La collecte automatique date de quelques semaines : la plupart des annonces sont apparues lors de la première collecte complète, et les variations de prix sont encore rares

(deux sur la dernière semaine). Les tendances temporelles et la saisonnalité de l'estival ne sont pas encore mesurables.

- **Dépendance aux sources.** Une modification de structure d'un site source interrompt sa collecte jusqu'à adaptation du collecteur. Les tests sur pages de référence la détectent, sans l'empêcher.
- **Géolocalisation approximative.** Une partie des annonces n'est située qu'au centre de leur quartier ou de leur ville. L'interface le signale, mais les distances aux points d'intérêt sont alors imprécises.
- **Fiabilité et standing simples.** M4 n'utilise que trois variables, et M5 repose sur des règles. Aucun des deux n'a pu être validé contre une vérité de terrain étiquetée.

9.4 Conclusion

ImmoInsight TN montre qu'il est possible de construire, à partir des seules annonces publiques, une lecture chiffrée, actualisée et compréhensible du marché immobilier tunisien. La plateforme collecte chaque jour les annonces de trois sources et en contrôle la qualité de manière traçable. Elle en retient 3 214, géolocalisées à 99,6 %, sur lesquelles elle estime prix, loyers et prix par nuit avec une erreur médiane de 21 à 29 %, toujours nettement inférieure à celle d'une méthode naïve et affichée telle quelle à l'utilisateur. Elle restitue ces informations par une carte, des statistiques régionales, des alertes de marché et un assistant qui ne répond qu'à partir des données.

Au-delà du résultat, le projet a confirmé quelques principes qui valent pour tout produit fondé sur la donnée : soigner la donnée avant l'algorithme, mesurer honnêtement contre une référence, montrer l'incertitude, et industrialiser tôt (tests, intégration continue, conteneurs, analyse de sécurité), pour que chaque correction reste acquise.

9.5 Perspectives

Données.

- Accumuler l'historique pour mesurer les tendances de prix par région et la saisonnalité de l'estival, et en tirer un indice de prix mensuel.
- Étendre la collecte aux régions sous-représentées et à de nouvelles sources, dans le respect de leurs conditions d'utilisation.
- Rapprocher, si des données ouvertes le permettent un jour, prix affichés et prix de transaction.

Modèles.

- Évaluer les modèles *dans le temps* (entraînement sur les annonces anciennes, test sur les récentes), plus proche de l'usage réel qu'un découpage aléatoire.
- Expliquer chaque estimation par les contributions de ses variables (valeurs de Shapley), devenu réaliste avec plus d'un millier d'annonces par segment.
- Remplacer les règles de M5 par un classifieur de texte en français, et enrichir M4 de variables textuelles et de l'historique de l'annonceur, une fois des exemples étiquetés réunis.

Produit.

- Déployer la plateforme en ligne (hébergement de conteneurs et base MongoDB gérée), avec HTTPS.
- Proposer des alertes personnalisées par courriel (zone, budget, type de bien).
- Ajouter un mode sombre et des tests automatisés de l'interface.
- Réduire la complexité des fonctions signalées par SonarQube.

Bibliographie

[1] S. Rosen. « Hedonic prices and implicit markets: product differentiation in pure competition ». *Journal of Political Economy*, 82(1):34–55, 1974.

[2] T. Chen et C. Guestrin. « XGBoost: A scalable tree boosting system ». *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, p. 785–794, 2016.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye et T.-Y. Liu. « LightGBM: A highly efficient gradient boosting decision tree ». *Advances in Neural Information Processing Systems 30*, 2017.

[4] D. H. Wolpert. « Stacked generalization ». *Neural Networks*, 5(2):241–259, 1992.

[5] D. Micci-Barreca. « A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems ». *ACM SIGKDD Explorations Newsletter*, 3(1):27–32, 2001.

[6] A. Z. Broder. « On the resemblance and containment of documents ». *Proceedings of Compression and Complexity of Sequences*, IEEE, p. 21–29, 1997.

[7] F. T. Liu, K. M. Ting et Z.-H. Zhou. « Isolation forest ». *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, p. 413–422, 2008.

[8] F. Pedregosa *et al.* « Scikit-learn: Machine learning in Python ». *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan et Y. Cao. « ReAct: Synergizing reasoning and acting in language models ». *International Conference on Learning Representations (ICLR)*, 2023.

[10] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda et T. Scialom. « Toolformer: Language models can teach themselves to use tools ». *Advances in Neural Information Processing Systems 36*, 2023.

[11] W3C. *Web Content Accessibility Guidelines (WCAG) 2.2*. Recommandation du W3C, 2023. `https://www.w3.org/TR/WCAG22/`

[12] Contributeurs d'OpenStreetMap. *OpenStreetMap*. Données sous licence ODbL. `https://www.openstreetmap.org`

[13] Documentations officielles : Scrapy (`https://docs.scrapy.org`), Playwright (`https://playwright.dev`), FastAPI (`https://fastapi.tiangolo.com`), Angular (`https://angular.dev`), Leaflet (`https://leafletjs.com`), MongoDB (`https://www.mongodb.com/docs`).